

NAME:TANKISO MASOEBE

ADVANCED DATABASE WK2

1. Distributed Database for a Global Retailer

A global retailer can use a distributed database to store and manage information across different countries and regions. Instead of keeping all data in one central location, the database is distributed across multiple servers in different locations. **Partitioning** can divide the data into smaller sections based on factors such as country, region, or store. For example, customer and sales data from Lesotho can be stored on servers in Southern Africa, while European data can be stored on European servers. This improves performance because users can access data from a nearby server. Replication can also be used to keep copies of important data on multiple servers, improving availability if one server fails. The distributed database allows the retailer to process transactions, manage inventory, track customers, and analyse sales efficiently. Overall, partitioning and replication make the system faster, more reliable, scalable, and suitable for supporting retail operations worldwide.

2. CAP Theorem Research

The CAP theorem states that a distributed database system cannot guarantee all three properties at the same time: **Consistency, Availability, and Partition Tolerance**. Consistency means every user sees the most recent data. Availability means every request receives a response, even when some servers fail. Partition tolerance means the system continues operating when communication between servers is interrupted. In a network partition, a system must choose between consistency and availability. For example, an online retailer may experience a network failure between two database servers. If it chooses consistency, some transactions may be temporarily rejected. If it chooses availability, transactions can continue, but users may temporarily see outdated information.

3. Cassandra Exploration

Apache Cassandra is a distributed NoSQL database designed to handle large amounts of data across multiple servers. For this practical, Cassandra can be installed using Docker and accessed through the Cassandra command line. A sample customers table can be created with columns such as customer_id, name, country, and email. The table can be partitioned using customer_id as the partition key. This allows Cassandra to distribute customer records across different nodes and retrieve data efficiently. For example, records with different customer IDs are stored in appropriate partitions. Partitioning improves scalability and performance because the workload can be shared across multiple servers. Cassandra also provides high availability and fault tolerance.

1. Run Cassandra with Docker

```
docker pull cassandra  
docker run --name cassandra -d cassandra
```

Check that it is running:

```
docker ps
```

2. Open Cassandra

```
docker exec -it cassandra cqlsh
```

3. Create a Keyspace

```
CREATE KEYSPACE retail  
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
```

Use it:

```
USE retail;
```

4. Create a Sample Table and Partition It

```
CREATE TABLE customers (  
    customer_id INT,  
    name TEXT,  
    country TEXT,  
    email TEXT,  
    PRIMARY KEY (customer_id)  
);
```

Here, **customer_id is the partition key**. Cassandra uses it to determine where each customer's data is stored.

5. Insert Sample Data

```
INSERT INTO customers (customer_id, name, country, email)  
VALUES (1, 'Tankiso', 'Lesotho', 'tankiso@example.com');
```

```
INSERT INTO customers (customer_id, name, country, email)  
VALUES (2, 'John', 'South Africa', 'john@example.com');
```

```
INSERT INTO customers (customer_id, name, country, email)  
VALUES (3, 'Mary', 'Botswana', 'mary@example.com');
```

6. Display the Data

```
SELECT * FROM customers;
```

Partitioning: Each `customer_id` acts as a partition key, allowing Cassandra to distribute customer records across nodes efficiently.

4. Consistency Model Analysis

Strong consistency ensures that every user or device accessing an IoT database always receives the most recent data. This is useful for critical systems such as industrial sensors, where incorrect or outdated information could cause problems. However, strong consistency can increase response time and reduce availability during network failures. Eventual consistency allows devices to continue operating and updating data even when some servers cannot communicate. The database becomes consistent after a period of time, making it faster and more available. For example, a smart home system can use eventual consistency for temperature readings. Overall, strong consistency provides accuracy, while eventual consistency provides better performance and availability.